IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS

DIGITAL ELECTRONICS AND COMPUTER ARCHITECTURE

Tuesday, 5 May 2026, 2:30 PM

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions.

Question One carries 40 marks. Question Two and Question Three carry 30 marks each.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : G.A. Constantinides, T. Clarke
Second Marker(s) : T. Clarke, G.A. Constantinides

© Imperial College London

DIGITAL ELECTRONICS AND COMPUTER ARCHITECTURE

1. a) Simplify the following Boolean expression using Boolean Algebra. At each step of your working, state the rule you are using to simplify (you may use Huntington's axioms and/or any of the rules named or numbered in the lecture notes).

$$(A+B)(A+\overline{B})(\overline{A}+C)$$

[4]

b) Simplify the Boolean expression $Y = \sum\{1,3,4,6,7\}$ using a Karnaugh map, leaving your answer in sum-of-products form. The inputs to this function are X_2, X_1, and X_0, in order from most- to least-significant input. Show all your working including the K-map groups used.

[4]

c) The SOP $F = AB + \overline{A}\,\overline{C}$ can result in a static hazard. Identify the input conditions under which it arises, and modify the SOP to avoid the hazard.

[3]

d) (i) Write the decimal values -7_{10} and 19_{10} in 8-bit two's complement. (ii) state the largest negative number and the largest positive number that can be represented in 8-bit two's complement.

[4]

e) For a 4K x 8 ROM, state the following quantities in bits (i) the width of the address bus, (ii) the width of the data bus, (iii) the total capacity of the ROM.

[3]

f) Define the setup and hold times of a D-type flip-flop.

[2]

```
MOV R1, #0
SUB R1, R1, R1
SBC R1, #0
MOV R0, #-100
```

Figure 1.1 ALU instructions

```
LSL R5, R4, #10
LSR R6, R4, #6
SUB R5, R5, R4
SBC R6, #0
```

Figure 1.2 ALU instructions with shifts

```
MOV R0, #0x40
EXT 0x30
STR R0, [0xF0]
MOV R1, #0x41
STR R0, [R1]
STR R1, [R1,#-1]
```

Figure 1.3 Memory store instructions

g) The EEP1 assembly code in Figure 1.1 executes starting at memory address 0. In the clock cycle immediately after the last instruction executes, state the values of registers R0, R1, and status flags N, Z, C, V.

[4]

h) The EEP1 assembly code in Figure 1.2 takes a 16-bit input in R4 and produces a 32-bit output in R6:R5. Find expressions for R6(i) and R5(i), after the execution of the two shift instructions, in terms of R4(i). Hence, or otherwise, determine an arithmetic expression for u(R6:R5) as a function of u(R4).

[6]

i) The EEP1 assembly code in Figure 1.3 writes to three memory locations. Construct a table showing the addresses written and the corresponding data values.

[6]

j) Illustrate the register values and written memory contents from the previous part with a pointer diagram.

[4]

2. This question relates to building a sequential circuit that has a single Boolean input A and a single Boolean output B.

Let $A[n]$ denote the input during clock cycle n and let $B[n]$ denote the output during clock cycle n. The circuit starts operating at cycle $n = 0$.

Let $\#v$ denote the number of 1s in a Boolean vector v.

The goal is to use FSM design methodology to design a circuit such that

- for $n = 0$, $B[n] = 0$,
- for $n = 1$, $B[n] = 1$ if and only if $\#(A[n], A[n-1]) \geq 2$,
- for $n \geq 2$, $B[n] = 1$ if and only if $\#(A[n], A[n-1], A[n-2]) \geq 2$.

a) State with justification whether this behaviour is implementable by a Moore machine.

[2]

b) State with justification the minimum number of states required to implement this behaviour.

[3]

c) Draw a state transition diagram with clearly labelled state names/numbers, transitions, inputs and outputs, implementing this behaviour.

[8]

d) Select a Boolean encoding for each of the states.

[2]

e) For the encoding you have selected, create Karnaugh maps for each next-state bit Q_i^+ in terms of the current-state bits Q and the single Boolean input A. Use the Karnaugh maps to derive minimal SOP expressions for each of Q_i^+, showing your working. Explicitly show and label groups in the K-map corresponding to products in your SOP.

[12]

f) For the encoding you have selected, create a Karnaugh map for the single output B in terms of the current state bits Q and the single Boolean input A. Use the Karnaugh map to derive a minimal SOP expression for B, showing your working. Explicitly show and label groups in the K-map corresponding to products in your SOP.

[3]

3. Figure 3.2 shows a schematic of the interrupt mechanism used in the EEP1X CPU. EEP1X is based on the EEP1 with added support for interrupts. EEP1X is different from the EEP1I processor studied in the labs. Assume that an interrupt request signal may be asserted by an external device, not shown here.

Interrupts require three additional instructions: SETI, CLRI and RETINT. They also require two hardware additions: a 16-bit register ISP and a 1-bit status flag FlagI.

FlagI allows software, via CLRI and SETI, to disable or enable interrupts. ISP is a stack pointer used only during interrupt service routines (ISRs). On entry to an ISR from the base program, the hardware swaps R7 with ISP so that the ISR uses a separate stack in EEP1 data memory with R7 as the stack pointer

Nested interrupts continue to use R7 as the stack pointer and therefore do not swap R7 and ISP.

The figure also shows the operations performed when an interrupt is taken and when the ISR terminates via the RETINT instruction.

a) Figure 3.1 shows the total T_{ISR} and prelude T_{PRE} times of four interrupt sources with the priorities shown. Calculate the worst-case response time for the priority-3 interrupt source, assuming interrupts are prioritised but not nested. Show your working.

[6]

b) Calculate the worst-case interrupt response time again, now assuming interrupts are nested. An ISR cannot be interrupted during its prelude, but may be pre-empted by a higher-priority interrupt once T_{PRE} time units have elapsed from the start of that ISR. Show your working.

[6]

c) The interrupt payload reads a 6-bit signed two's-complement number from a memory-mapped input device and adds it to a cumulative signed total S, stored in a 16-bit register. Suppose that N such numbers are accumulated. Calculate the maximum value of N for which overflow in S is impossible, regardless of the input values. Generalise your result to the accumulation of n-bit signed two's-complement numbers.

[6]

d) The interrupt stack is an upwards-growing full-SP stack. Interrupt entry and exit code must save and later restore registers R0, R1 and R2 on this stack.

The entry code must enable interrupts. The exit code must first disable interrupts and then return from the interrupt. Write the sequences of EEP1X instructions for the interrupt entry code and the interrupt exit code. The RETINT instruction must be the final instruction of the exit code.

[6]

e) Suppose now that, in order to decrease T_{PRE}, interrupts are enabled before the start of the entry code. State, with reasons, the ordering constraints on the stack-pointer update and the register-store instructions that ensure correct operation of the interrupt stack even if another interrupt occurs during execution of the entry code.

Illustrate your answer with a short example of a correct entry instruction sequence.

[6]

Priority	T_{PRE}	T_{ISR}
4	10	20
3	10	40
2	10	80
1	10	160

Figure 3.1 Interrupt Timing

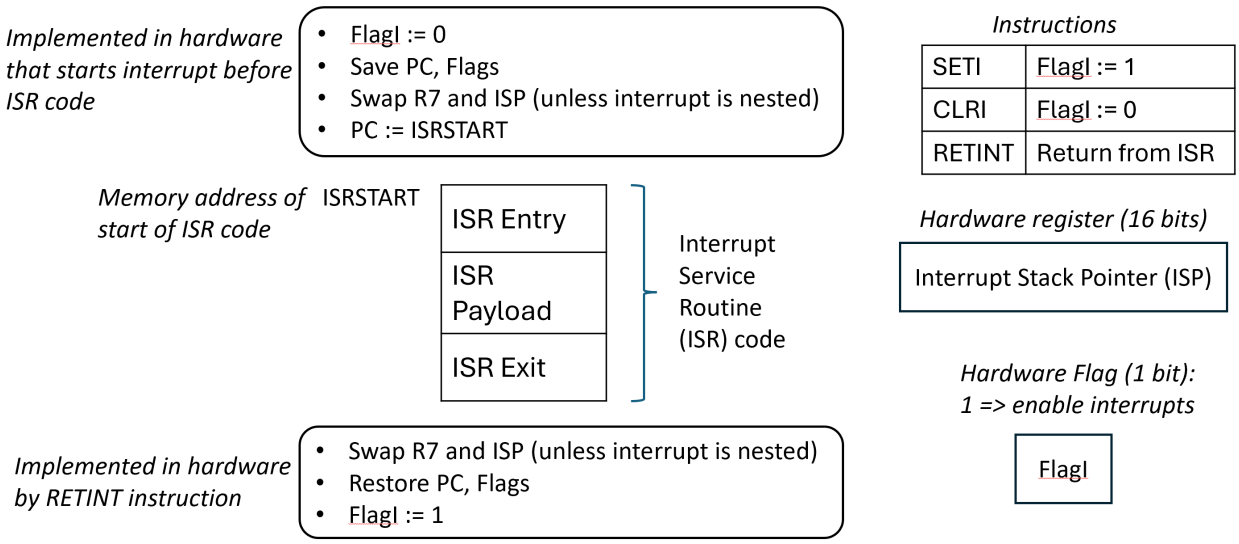


Figure 3.2 Interrupt hardware and software

DECA Spring Term Exam Notes 2026

(These notes will be distributed in the Exam printed greyscale 2 slides per side)

- The EEP1 CPU is a 16 bit Harvard architecture, with 16 bit addresses and word addressing for both data and code, hence 2^{17} bytes of instruction memory and 2^{17} bytes of data memory.
- At start of execution all EEP1 registers and flags are zero
- All EEP1 instructions take 1 clock cycle to execute

EEP1 Machine Code Overview

SHIFTOPC(1:0)	Shift
0	LSL
1	LSR
2	ASR
3	XSR

ALUOPC	INS(8)=0	INS(8)=1
0	MOV[1] Ra, Rb	MOV Ra, #IMM
1	ADD Rc, Ra, Rb	ADD Ra, #IMM
2	SUB Rc, Ra, Rb	SUB Ra, #IMM
3	ADC Rc, Ra, Rb	ADC Ra, #IMM
4	SBC Rc, Ra, Rb	SBC Ra, #IMM
5	AND Rc, Ra, Rb	AND Ra, #IMM
6	CMP[1] Ra, Rb	CMP Ra, #IMM
7	Shift Ra, Rb, #SCNT (see SHIFTOPC for Shift)	

[1]Instruction does not use Rc, this field is (0)

legend	Meaning
Imm4	4 bit unsigned immediate
Imms5	5 bit signed immediate
Imms8	8 bit signed immediate
(0)	Must be 0 for current instructions, non-zero values are reserved for expansion

EEP1 machine code	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	ALU	0	ALUOPC=7			a			Shift-opc(1)	b			Shift-opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c			(0)	
									1	Imms8 (IMM)							
	Load / Store	1	0	MOPC	(0)	a			0	b			Imms5 (OFFSET)				
									1	Imms8 (IMM)							
	Jump	1	1	0	0	JMPOPC				Imms8 (JOFFSET)							
	EXT	1	1	0	1	(0)				Imms8 (IMM)							
	Not used	1	1	1	X	X											

EEP1 ALU Instructions

ALUOPC	INS(8)=0	INS(8)=1
0	MOV[1] Ra, Rb	MOV Ra, #IMM
1	ADD Rc, Ra, Rb	ADD Ra, #IMM
2	SUB Rc, Ra, Rb	SUB Ra, #IMM
3	ADC Rc, Ra, Rb	ADC Ra, #IMM
4	SBC Rc, Ra, Rb	SBC Ra, #IMM
5	AND Rc, Ra, Rb	AND Ra, #IMM
6	CMP[1] Ra, Rb	CMP Ra, #IMM
7	Shift Ra, Rb, #SCNT (see SHIFTOPC for Shift)	

[1]Instruction does not use Rc, this field is (0)

SHIFTOPC(1:0)	Shift
0	LSL
1	LSR
2	ASR
3	XSR

Legend	Meaning
Imm4	4 bit unsigned immediate
Imms8	8 bit signed immediate
(0)	Must be 0 for current instructions, non-zero values are reserved for expansion

	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EEP1 machine code	ALU	0	ALUOPC=7			a			Shift-opc(1)	b			Shift-opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c			(0)	
									1	Imms8 (IMM)							

EEP1 ALU instructions: detail

OPCALU	Mnemonic	ALU.Out: INS(8)=0	ALU.Out: INS(8) = 1	Write FlagV & FlagC[1]	Write register	
0	MOV	Rb	Imm	No	Ra	move
1	ADD	Ra + Rb	Ra + Imm	Yes	Rc (Ra if INS(8)=1)	arithmetic
2	SUB	Ra - Rb	Ra – Imm	Yes	Rc (Ra if INS(8)=1)	
3	ADC	Ra + Rb + C	Ra + Imm + C	Yes	Rc (Ra if INS(8)=1)	
4	SBC	Ra – Rb + (C-1)	Ra – Imm + (C-1)	Yes	Rc (Ra if INS(8)=1)	
5	AND	Ra & Rb	Ra & Imm	No	Rc (Ra if INS(8)=1)	"bitwise" logical
6	CMP	Ra - Rb	Ra - Imm	Yes	no	comparison
7	LSR, ASR, XSR, LSL	depends on SHIFTOPC		Yes[2]	Ra	shift

Flag	
FlagN	z(ALU.Out) < 0
FlagZ	ALU.Out = 0
FlagC	carry
FlagV	signed overflow

&	16 bit C++ bitwise logical AND operator

FlagN & **FlagZ** are written by **all** ALU instructions

Flags are only written by ALU instructions

[1]FlagC

- 1 bit flip-flop in CPU used to store carry for multi-word addition and shifts
- Holds previous value unless written

C = FlagC.Q

[2]Shift instructions write FlagC with *last bit shifted out*, FlagV is written to an undefined value.

EEP1 Memory Instructions

LDR	**Load Register**
STR	**Store Register**

INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	0	MOPC	(0)	a			0	b			Imms5 (OFFSET)				
								1	Imms8 (IMM)							

Field	Limits	Allowed Syntax
Imms5	-16 .. +15	`#0xA, #12, #-0x10, #-15`
Imms8	-128 .. 127	`#0x34, #100, #-33`
a, b	0..7	`R0, R5, R7`

Type	MOPC	I(8)	Assembly Syntax	Examples	Effective Address	Operation
register offset load	0	0	`LDR Ra, [Rb, #Imms5]`[3] `LDR Ra, [Rb]`[2]	`LDR R4,[R6,#-12]` `LDR R2, [R7]`	Rb+Imms5[1] Rb	Ra := mem[Rb + Imms5[1]] Ra := mem[Rb]
direct load	0	1	`LDR Ra, [#Imms8]`[4]	`LDR R0,[#220]`	Imms8[1]	Ra := mem[Imms8[1]]
register offset store	1	0	`STR Ra, [Rb, #Imms5]`[3]	`STR R6, [R6, #9]`	Rb+Imms5[1]	mem[Rb + Imms5[1]] := Ra
direct store	1	1	`STR Ra, [#Imms8]`[4]	`STR R0, [#0x66]`	Imms8[1]	mem[Imms8[1]] := Ra

[1] Imms5 is a 5 bit 2's complement signed number sign extended to 16 bits, Imms8 is 8 bits sign extended to 16 bits. Imms8=255 is the same as Imms8=-1.
[2] Special case: assembly syntax allows Imms5 offset to be omitted in which case it has value 0 in the machine code.
[3] Register offset addressing. [4] Direct addressing.

EEP1 Jump instructions

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	0	JMPOPC				Imms8 (JOFFSET)							

JMPOPC(3:1)	NZCV condition	Condition on data for jump with JMPOPC(0)=0	JMPOPC(0) = 0	JMPOPC(0) = 1[3]
0	1	always	**JMP** (Always jump)	**NOP** (Never jump)
1	Z	result = 0	**JEQ** (Equal)	**JNE** (Not Equal)
2	C	C=1 ≡ u(Ra) ≥ u(Op)[1]	**JCS** (Carry Set / Unsigned Higher or Equal)	**JCC** (Carry Clear / Unsigned Lower)
3	N	z(result) < 0	**JMI** (Minus)	**JPL** (Plus)
4	$\bar{N} \oplus V$	z(Ra) ≥ z(Op)[1]	**JGE** (Signed Greater or Equal)	**JLT** (Signed less than)
5	$(\bar{N} \oplus V).\bar{Z}$	z(Ra) > z(Op)[1]	**JGT** (Signed Greater Than)	**JLE** (Signed Less than or Equal)
6	$C.\bar{Z}$	u(Ra) > u(Op)[1]	**JHI** (Unsigned Higher)	**JLS** (Unsigned Lower or Same)
7	1	always	***JSR*** *(R7[4] := PC + 1, PC := PC + JOFFSET)*	***RET*** *(PC := R7)*

[1] **JGE,JGT,JHI, JCS** jump conditions jump based on the flag values. The table column 3 shows the comparison of **Ra** and **Rb** after a previous `SUB Ra, Rb` instruction which delivers those NZCV values, or the condition on an ALU result which delivers the required NZ values.

[2] **JSR** and **RET** always jump and are used to *jump to* and *return from* subroutine see notes. **RET** does not use **Joffset**.

[3] **JMPOPC(0) = 1** inverts all conditions, except for JSR/RET

[4] Hardware is simpler because **JSR**/**RET** write/read R7 since **Ra** is normally read and written and for these instructions **a** = **JMPOPC(3:1)** = 7

Jump Instructions	Assembly	Operation	Examples
All except RET, JSR	JEQ JOFFSET; JEQ SYMBOL	PC := PC + JOFFSET if condition is true	`JMP −1 ; JGT LOOP ; JSR SUB1`
RET	RET	PC := R7	`RET`
JSR	JSR JOFFSET	R7 := PC+1; PC := PC + JOFFSET	`JSR 10`

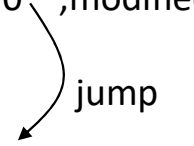
EXT instruction

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	1	(0)				Imm8 (IMM)							

Assembly example

Memory address

0x0100 EXT 0x23
0x0101 JMP 0xF0 ;modified jump instruction



0x24F1 …

0x2300 *EXT IMM8 = 0x23*
0x00F0 *JMP IMMS8 = 0xF0*
0x0101 *Address (PC) of JMP instruction*
0x24F1 *Jump target address*

- EXT modifies the instruction immediately after it, replacing bits (15:8) of Imm16 (otherwise from Imms8 sign extension) with IMM rom the EXT instruction
- NB this means the 8 Imms8 bits from the modified instruction are not sign extended because the extended part is replaced.
- Applies to:
  - Jump instructions,
  - **Direct** memory instructions
  - ALU instructions with immediate operand

EEP1 Subroutine Call Protocol determines how registers are used by subroutines

- The EEP1 subroutine stack is a *downwards growing, empty stack pointer* stack using R6 as stack pointer.
- The protocol fixes the usage of each register for all subroutines and determines what must be saved or restored on subroutine entry/exit.
  - **Scratch** registers (can be changed by any subroutine, not saved/restored at entry/exit)
  - **Persistent** registers (must be saved/restored at entry exit by callee subroutine on callee's stack frame if used)
  - **Stack Pointer**
  - **Return Address**
- All ISAs use protocols similar to this so that compiled code can be turned into subroutines
- Scratch registers are also used if needed for 1st three subroutine inputs and outputs

EEP1 standard subroutine call protocol (the *caller* calls the *callee*)			
Register	Content Type	Saved by callee on subroutine entry/exit if used	Persistent across subroutine calls, can be used for storage by caller
R7	**Return Address**	**yes**	**no**
R6	**Stack Pointer (SP)**	**n/a**	**yes, no***
R5	persistent	yes	yes
R4	persistent	yes	yes
R3	persistent	yes	yes
R2	scratch (or input 3, or output)	no	no
R1	scratch (or input 2, or output)	no	no
R0	scratch (or input 1, or output)	no	no

*R6 can't be used for storage by caller, because it must always contain the stack pointer

Notation for DECA Spring 2025 (1)

- **High, low.** Synonym for digital logic values 1, 0.
- **X.** *don't care* value that does not affect operation
- **Clocked (sequential) logic.** Logic with edge triggered flip-flops. All clocks are assumed connected together to a single global clock.
- **Combinational logic.** Logic with gates and no flip-flops, also no *latches* AKA RS flip-flops, nor logic with feedback connections.
  - NB we will not use *latches* in this module, therefore combinational logic loops (that might make latches) are also not allowed. Every such loop must be broken by a D -> Q flip-flop path, or equivalent memory path.
- **Asynchronous (synchronous) RAM.**
  - The *output* is valid in the same cycle (the next cycle) as the *address*. In both cases the write operation is synchronous, and locations change to a newly written value in the cycle after that in which WEN is 1 and ADDR, DIN specify the written location and value.
- **Register EN** – enable input, if 1 then something is written
  - Named WEN on RAMs and register files
- **Decoder EN** – enable input, if 0 then all outputs are 0
  - Note differing use of EN on clocked and combinational logic.
- **A := B.** Notation to set register A equal to B on active clock edge at end of current clock cycle. B can be a register or a number.
- **Mem[a]** – memory *location* with *word* address a. **Mem[a] := B** – set memory location address a to contents of B (or B – if B is a number).
- **Mem_{32}[a] (Mem_{16}[a]), Mem_8[a]** word and byte locations in a 32 (16) bit system with byte addressing such as ARM. Word or byte memory read and write are defined with **:=** notation as above for Mem[a]. Byte memory transfers are as defined in notes.
- **MUX.SEL** – signal SEL on block MUX
- **MUX.N** input – selects which of MUX data inputs passes to output. Data inputs are numbered according to their N value.
- **IR(7)** – *selected bit 7* of binary data or register IR. Bits of n bit bus are numbered *little-endian* with bit 0 being LSB, bit n-1 MSB.
- **IR(3:0)** – *slice* of binary data or register IR. Bits from 3 to 0 inclusive.

Notation for DECA Spring 2025 (2)

- **Byte.** 8 bits of data. Possibly interpreted as unsigned (0..255) or signed (-128..127) number. See u(A) & z(A) definitions
- **Word.** Binary data in one CPU memory location or CPU data register. Typical values 8, 16, 32, 64 bits. Sometimes interpreted as unsigned or signed number.
- **MSW** (**LSW**) Most (Least) Significant Word, when words are joined to make a binary number with more bits
- **MSB (LSB). MS byte, LS byte**. The most significant or least significant bit or byte respectively of a binary word.
- **R1:R2**. The binary value made from the bits of R1 (MSBs) and R2 (LSBs). If R1, R2 are 16 bit: (R1:R2)(31:16) = R1 and (R1:R2)(15:0) = R2
- $\mathbf{A_N}$ - Value of the **A** signal in cycle N. NOT **A(N)** the Nth bit of A!
- **0xA30, 0b010**, hex, binary representations of numbers.
- **R0, Rn** CPU register 0, n (e.g. n = 0..3 or n = 0...15). CPU registers have numbers R0, R1, ... For a register file with named registers REGn REG[i] indicates register REGi.
- **u(A), z(A),** Respectively the *unsigned* and *signed* numeric interpretation of signal A. A must have a known width *n*, in which case u(A) and z(A) differ only when the sign bit A(n-1) is 1. *Bit slicing* is allowed on integers derived from u(), z(): (u(P(n-1:0))+u(Q(n-1:0)))(n-1:0) is the n Sum bits of a binary adder with n-bit inputs P, Q. The carry output bit from this could be written as: (u(P(n-1:0))+u(Q(n-1:0)))(n)
- $\mathbf{A_S}$ = A(n-1), the most significant (sign) bit of A(n-1 : 0).
- **A'** If A= A(n-1 : 0) has n bits, **A'** = A(n-2 : 0). If used in a numeric context this is interpreted as u(A'): an unsigned integer.
- $\boldsymbol{m}$ **mod** $\boldsymbol{n}$. The same as `m % n` in C++. $\boldsymbol{m}$ **mod** $\mathbf{2^n}$ is the unsigned value of the LS 8 bits of m. If m is negative this is two's complement: -1 `mod` $2^8 = 255$; 260 `mod` $2^8 = 4$; x `mod` 2^n is x truncated to its n least significant bits. Division is assumed integer unless otherwise stated.